CUDA Image Inversion Program

Aim: To invert the colours of an image using CUDA parallel processing. Each pixel channel is transformed using **output = 255 - input**.

1. CUDA C++ Program (image_invert.cu)

```
#include <opencv2/opencv.hpp>
#include <cuda_runtime.h>
#include <iostream>

using namespace cv;
using namespace std;

__global__ void invertKernel(unsigned char *input,
                             unsigned char *output,
                             int totalBytes) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < totalBytes) {
        output[idx] = 255 - input[idx];
    }
}

int main() {
    Mat image = imread("flower.jpg", IMREAD_COLOR);

    if (image.empty()) {
        cout << "Image not found!" << endl;
        return -1;
    }

    int totalBytes = image.rows * image.cols * image.channels();

    unsigned char *d_input;
    unsigned char *d_output;

    cudaMalloc((void**)&d_input, totalBytes);
    cudaMalloc((void**)&d_output, totalBytes);

    cudaMemcpy(d_input, image.data, totalBytes,
               cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid =
        (totalBytes + threadsPerBlock - 1) / threadsPerBlock;

    invertKernel<<<blocksPerGrid, threadsPerBlock>>>(
        d_input, d_output, totalBytes
    );

    cudaDeviceSynchronize();

    Mat output(image.rows, image.cols, image.type());

    cudaMemcpy(output.data, d_output, totalBytes,
               cudaMemcpyDeviceToHost);

    imwrite("inverted.jpg", output);

    cudaFree(d_input);
    cudaFree(d_output);

    cout << "Image inversion completed." << endl;
    cout << "Output saved as inverted.jpg" << endl;

    return 0;
}
```

2. Run Command (CUDA C++)

With NVIDIA CUDA Toolkit and OpenCV development files installed, compile with a command similar to:

```
nvcc image_invert.cu -o image_invert `pkg-config --cflags --libs opencv4`
./image_invert
```

Place **flower.jpg** in the same directory before running. The output is saved as **inverted.jpg**. The exact compile command may vary by operating system and OpenCV installation.

3. Python CUDA Program (Google Colab)

Select a GPU runtime in Google Colab (Runtime → Change runtime type → GPU), then run the following cells in order.

Cell 1 — Upload image

```
from google.colab import files
uploaded = files.upload()
```

Cell 2 — Import libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numba import cuda
```

Cell 3 — Read image

```
img = cv2.imread("flower.jpg")

if img is None:
    raise FileNotFoundError("Image not found!")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
print("Image shape:", img_rgb.shape)
```

Cell 4 — CUDA kernel

```
@cuda.jit
def invert_image(input_img, output_img):
    x, y, c = cuda.grid(3)

    if (x < input_img.shape[0] and
        y < input_img.shape[1] and
        c < input_img.shape[2]):
        output_img[x, y, c] = 255 - input_img[x, y, c]
```

Cell 5 — Execute kernel

```
output = np.zeros_like(img_rgb)

threadsperblock = (16, 16, 3)

blockspergrid_x = (img_rgb.shape[0] + 15) // 16
blockspergrid_y = (img_rgb.shape[1] + 15) // 16
blockspergrid_z = 1

blockspergrid = (
    blockspergrid_x,
    blockspergrid_y,
    blockspergrid_z
)

invert_image[blockspergrid, threadsperblock](img_rgb, output)
cuda.synchronize()

print("Image inversion completed.")
```

Cell 6 — Display images

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(output)
plt.title("CUDA Inverted Image")
plt.axis("off")

plt.show()
```

Cell 7 — Save and download

```
output_bgr = cv2.cvtColor(output, cv2.COLOR_RGB2BGR)
cv2.imwrite("inverted.jpg", output_bgr)

from google.colab import files
files.download("inverted.jpg")
```

4. Expected Result

The program displays the original image beside its colour-inverted version. Each channel value is replaced by 255 minus its original value. The processed image is saved as **inverted.jpg**.

Notes

• The C++ version requires an NVIDIA CUDA-capable GPU, CUDA Toolkit, and OpenCV development libraries.
• The Python version requires a compatible NVIDIA GPU runtime and a Numba version that supports the runtime.
• If the Colab environment reports that Numba is missing, install it in a separate cell with **!pip install numba**, then restart the runtime if prompted.